UNIVERSIDAD NACIONAL DEL ALTIPLANO PUNO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS

ESCUELA PROFESIONAL INGENIERÍA DE SISTEMAS



CURSO:

Programación Orientada a Objetos-II

TRABAJO:

# Revisión de POO I: Diagnostico, Refactorización y Calidad de Código

(Parte 1)

PRESENTADO POR:

Melvin Ronald Choque Chaiña

DOCENTE:

Mg. Aldo Hernan Zanabria Galvez

FECHA:

03/09/2026

PUNO-PERÚ

# Contenido

INTRODUCCIÓN ..... 3

1. Desarrollo de Actividades ..... 4

1.1. Actividad 1: Diagnostico Identificación de problemas en código existente ..... 4

1.2. Actividad 2: Refactorización en Python Misma lógica, mejor diseño ..... 5

1.2.1. Código Refactorizado Python ..... 6

1.3. Actividad 3: Equivalente en C++17 Encapsulamiento y Validación ..... 8

1.4. Actividad 4: Cohesión y Acoplamiento – Análisis Comparativo ..... 10

1.5. Actividad 5: Inicio del Proyecto Integrador – Propuestas y primeras clases 11

2. Preguntas de Reflexión ..... 12

# INTRODUCCIÓN

En el desarrollo de software moderno, la calidad de código y la arquitectura de diseño son factores determinantes para garantizar la mantenibilidad, escalabilidad y legibilidad de los sistemas a lo largo del tiempo. La programación Orientada a Objetos proporciona un paradigma sólido, estructurado en cuatro pilares fundamentales: encapsulamiento, herencia, polimorfismo y abstracción, sin embargo, la implementación inadecuada de estos conceptos suele generar antipatrones de diseño como las clases monolíticas que mezclan responsabilidades de dominio, persistencia, notificación y presentación, incrementando al acoplamiento y reduciendo la cohesión.

El presente informe corresponde a la Práctica Diagnóstica de la Semana 1(parte 1) de la asignatura Programación Orientada a Objetos II de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional del Altiplano. Esta experiencia tiene como propósito evaluar y consolidar las competencias previas de modelado y refactorización mediante el análisis crítico de un sistema en Python 3 y C++17

A lo largo de este documento, se aborda el diagnostico de un gestor académico defectuoso, identificando sus olores de diseño y aplicando técnicas de refactorización incremental. Se presenta la descomposición modular hacia la arquitectura de responsabilidades separadas, validada a través de pruebas unitarias, caracterización y respaldada por métricas comparativas de cohesión y acoplamiento. Asimismo, se establece el punto de partida y la propuesta inicial del Proyecto Integrador del Semestre.

# 1. Desarrollo de Actividades

## 1.1. Actividad 1: Diagnostico Identificación de problemas en código existente

Analice el siguiente código Python con problemas de diseño. No lo ejecute aún primero identifique todos los defectos de calidad que pueda encontrar.

```
# codigo_problematico.py – identifique todos los problemas de diseño class est: def
__init__(self, n, a, p): self.n = n self.a = a self.p = p
self.h = [] def add(self, c, n): self.h.append([c, n]) def ppa(self):
if len(self.h) == 0: return 0 t = 0 for i in self.h:
t = t + i[1] return t / len(self.h) def pr(self): print(self.n,
self.a, self.p, self.ppa()) for i in self.h: print(i) e1 =
est("Juan Mamani", 20210500, "Ing. Sistemas") e1.add("POO II", 16) e1.add("BD I", 14)
```

Complete la tabla de análisis de calidad

| Defecto detectado | Línea(s) | ¿Cómo lo corregirías? |
|---|---|---|
| Poco Claro y sin PEP 8: Múltiples instrucciones amontonadas en una sola línea mediante uso de “;” o concatenación directa. | Líneas 1-8 | Separar cada instrucción en su propia línea respetando los saltos de línea e indentación estándar de Python |
| Nombres no descriptivos: Uso de identificadores crípticos | Líneas 1,2,3,4,5,6 | Renombrar utilizando la convención PascalCase para la clase (Estudiante) y Snake_case descriptivo para variables y métodos nombre, código, carrera, historial_notas, agregar_notas, calcular_promedio |
| Violación de Responsabilidad Única: El método pr() mezcla la obtención de datos con | Líneas 6-7 | Reemplazar el método pr() por el método especial __str__() o __repr__() para que la clase solo retorne la representación en texto del objeto sin imprimirla directamente. |

| | | |
|---|---|---|
| la impresión por consola mediante print() | | |
| Estructura de datos frágil: Representar cada curso y nota como una lista anidada [c, n] accesible únicamente por índices numéricos (i[1]). | Líneas 3,5,7 | Utilizar diccionarios (ej. {“curso”: c, “nota”: n}), tuplas con nombre (namedtuple) o una clase secundaria Curso. |
| Falta de Encapsulamiento: Las variables de instancia (h, n, a, p) son totalmente publicas y modificables sin control desde afuera | Líneas 2,3 | Definir como atributo como protected/private (self._historial_notas) y proveer métodos para interactuar con él. |
| Falta de Validación de Tipos y Rangos: add() acepta cualquier valor para las notas, sin verificar que sea un numero ni que esté en el rango permitido | Línea 3 | Incluir comprobaciones dentro del método de registro por ejem:<br><br>if not ( 0 <= nota <= 20): raise ValueError(…) |

## 1.2. Actividad 2: Refactorización en Python Misma lógica, mejor diseño

Reescriba el código de la Actividad 1 aplicando todas las correcciones identificadas. El resultado debe:

- Tener nombres descriptivos para la clase, atributos y métodos (Snake_case para Python).
- Usar atributos privados (_nombre, _codigo) con propiedades @property para acceso controlado.
- Separar la lógica de negocio (calcular _ppa) de la presentación (mostrar)
- Reemplazar listas anidadas [c, n] por un dataclass Nota(curso: str, calificación: float).
- Validar que la calificación esté en [0,20] al agregar una nota.

### 1.2.1. Código Refactorizado Python

Practicasem1.py

Practicasem1.py > ...

```python
# SU SOLUCIÓN REFACTORIZADA — Práctica S1, Actividad 2
from dataclasses import dataclass


@dataclass
class Nota:
    curso: str
    calificacion: float


class Estudiante:
    def __init__(self, nombre: str, codigo: int, escuela: str):
        self._nombre = nombre
        self._codigo = codigo
        self._escuela = escuela
        self._historial_notas: list[Nota] = []

    @property
    def nombre(self) -> str:
        return self._nombre

    def agregar_nota(self, curso: str, calificacion: float) -> None:
        if not (0 <= calificacion <= 20):
            raise ValueError(f"La calificación debe estar en el rango [0, 20]. Valor recibido: {calificacion}")

        self._historial_notas.append(Nota(curso, calificacion))

    def calcular_ppa(self) -> float:
        if not self._historial_notas:
            return 0.0

        suma_calificaciones = sum(nota.calificacion for nota in self._historial_notas)
        return suma_calificaciones / len(self._historial_notas)

    def __str__(self) -> str:
        lineas = [
            f"Estudiante: {self._nombre} | Código: {self._codigo} | Escuela: {self._escuela}",
            "Notas:"
        ]
        for nota in self._historial_notas:
            lineas.append(f"  - {nota.curso}: {nota.calificacion}")
        return "\n".join(lineas)

```

## Análisis del Código

El código implementa una solución orientada a objetos que corrige los problemas de diseño originales mediante las siguientes estructuras:

- Modelo de datos (Nota): Se utiliza **@dataclass** para definir de forma compacta y legible la entidad **Nota,** Reemplaza las listas anidadas **[c, n]** anteriores, permitiendo acceder a **nota.curso** y **nota.calificacion** mediante atributos claros.
- Encapsulamiento y Ocultamiento de Datos: La clase **Estudiante** almacena sus variables con el prefijo de guion bajo (**_nombre, _codigo, _escuela, _historial_notas**). Esto indica que son atributos internos protegidos.
- Acceso Controlado (**@property**): Se expone la propiedad **nombre** en modo de solo lectura mediante el decorador **@property,** impidiendo modificaciones directas no deseadas sobre el atributo privado desde el exterior.
- Validación de Dominio: El método **agregar_nota** valida que **0 <= calificación <= 20.** Si recibe un valor fiera de este rango, interrumpe el flujo lanzando una excepción **ValueError.**
- Separación de Responsabilidades:
  - **calcular_ppa()** se encarga de la lógica matemática sumando las notas y dividiendo entre el total de asignaturas. Protege el cálculo contra errores devolviendo **0.0** si la lista está vacía.
  - **__str__()** maneja de manera exclusiva la representación textual del objeto en formato multilínea, eliminando cualquier **print()** directo dentro de la lógica del objeto.

## Prueba del código:

```
44
45  # Prueba — debe ejecutar sin errores:
46  if __name__ == "__main__":
47      e = Estudiante("Juan Carlos Mamani Quispe", 20210500, "Ing. Sistemas")
48      e.agregar_nota("POO II", 16)
49      e.agregar_nota("Base de Datos I", 14)
50      print(e)
51      print(f"PPA: {e.calcular_ppa():.2f}")
52
53      try:
54          e.agregar_nota("Redes", 25)
55      except ValueError as ex:
56          print(f"Error capturado correctamente: {ex}")
```

Se diseñó un bloque de ejecución principal (**if __name__== "__main__":**) para verificar el correcto funcionamiento del modelo refactorizado, la precisión del cálculo del PPA y la efectividad del control de excepciones

Salida:

```
PS D:\2026-2\PROGRAMACIÓN ORIENTADA A OBJETOS 2\PRACTICAS SEMANALES  POO2\SEMANA 1\CODIGOS PRACTICAS>p
ython -u "d:\2026-2\PROGRAMACIÓN ORIENTADA A OBJETOS 2\PRACTICAS SEMANALES  POO2\SEMANA 1\CODIGOS PRAC
TICAS\Practicasem1.py"
Estudiante: Juan Carlos Mamani Quispe | Código: 20210500 | Escuela: Ing. Sistemas
Notas:
  - POO II: 16
  - Base de Datos I: 14
PPA: 15.00
Error capturado correctamente: La calificación debe estar en el rango [0, 20]. Valor recibido: 25
PS D:\2026-2\PROGRAMACIÓN ORIENTADA A OBJETOS 2\PRACTICAS SEMANALES  POO2\SEMANA 1\CODIGOS PRACTICAS>
```

## 1.3. Actividad 3: Equivalente en C++17 Encapsulamiento y Validación

Implemente la clase Estudiante en C++17 con encapsulamiento estricto, equivalente a la Actividad 2, Compile con g++-std=c++17-wall-o estudiante main.cpp.

main.cpp 1

main.cpp > Estudiante > calcularPPA() const

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
#include <utility>
#include <numeric>

struct Nota {
    std::string curso;
    float calificacion;
};

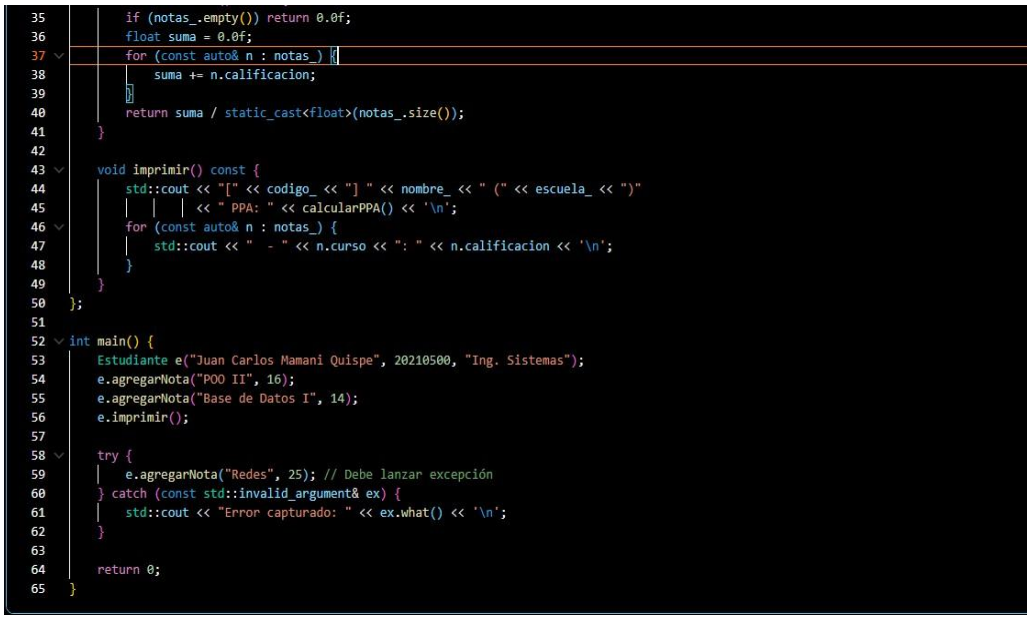
class Estudiante {
private:
    std::string nombre_;
    int codigo_;
    std::string escuela_;
    std::vector<Nota> notas_;

public:
    Estudiante(std::string nombre, int codigo, std::string escuela)
        : nombre_(std::move(nombre)), codigo_(codigo), escuela_(std::move(escuela)) {}

    const std::string& getNombre() const { return nombre_; }
    int getCodigo() const { return codigo_; }

    void agregarNota(const std::string& curso, float cal) {
        if (cal < 0.0f || cal > 20.0f) {
            throw std::invalid_argument("La calificacion debe estar en el rango [0, 20].");
        }
        notas_.push_back(Nota{curso, cal});
    }

    float calcularPPA() const {
```



```cpp
        if (notas_.empty()) return 0.0f;
        float suma = 0.0f;
        for (const auto& n : notas_) {
            suma += n.calificacion;
        }
        return suma / static_cast<float>(notas_.size());
    }

    void imprimir() const {
        std::cout << "[" << codigo_ << "] " << nombre_ << " (" << escuela_ << ")"
                  << " PPA: " << calcularPPA() << '\n';
        for (const auto& n : notas_) {
            std::cout << "  - " << n.curso << ": " << n.calificacion << '\n';
        }
    }
};

int main() {
    Estudiante e("Juan Carlos Mamani Quispe", 20210500, "Ing. Sistemas");
    e.agregarNota("POO II", 16);
    e.agregarNota("Base de Datos I", 14);
    e.imprimir();

    try {
        e.agregarNota("Redes", 25); // Debe lanzar excepción
    } catch (const std::invalid_argument& ex) {
        std::cout << "Error capturado: " << ex.what() << '\n';
    }

    return 0;
}
```

## Análisis del código:

El programa traslada la lógica de la actividad 2 a C++17 aplicando principios avanzados de Orientación a Objetos y gestión eficiente de memoria:

- Estructura de Datos Liviana (**struct Nota**)**:** Define un agregador de datos público de solo lectura para almacenar el par **curso y calificación.** Al ser una estructura simple optimiza el acceso y reduce la sobrecarga en comparación con clases complejas.
- Encapsulamiento Estricto: La clase **Estudiante** define sus miembros de datos como privados por defecto (**nombre_, código_, escuela_, notas_**). El uso del sufijo "_" sigue las convenciones estándar de C++ para diferenciar atributos de miembros locales.
- Gestión Eficiente de Memoria:
  - ❖ Semántica de Movimiento (**std::move**): En el constructor, los objetos **std::string** se transfieren mediante movimiento para evitar realizar copias costosas en la pila o el montón.
  - ❖ Paso por Referencia Constante (**const std::string** ): Los métodos reciben y retoman cadenas mediante referencias constantes, lo que elimina duplicaciones de memoria innecesaria durante su ejecución.
- Garantía de Inmutabilidad (**const** method): Los métodos **get.Nombre(), getCodigo(), calcularPPA()** e **imprimir()** están marcados explícitamente con el

calificador **const** al final. Esto asegurar al compilador que la ejecución de estos métodos no alterará el estado interno del objetos.

- Validación de Dominio y Excepciones Estándar: El método **agregarNota()** evalúa que la calificación se mantenga en el intervalo [0,20]. Si la regla se viola, interrumpe el flujo normal arrojando un objeto **std::invalid_argument** desde la librería estándar **<stdexcept>.**
- Cálculo Seguro del Promedio:
    - Verificar primero si el vector **notas_** está vacío usando. empty**()** para prevenir una excepción de división entre cero.
    - Realiza una conversión explicita de tipos mediante **static_cast<float>(notas_.size())** para prevenir advertencias o warnings de pérdida de precisión por parte del compilador.

Salida

```
[20210500] Juan Carlos Mamani Quispe (Ing. Sistemas) PPA: 15
  - POO II: 16
  - Base de Datos I: 14
Error capturado: La calificacion debe estar en el rango [0, 20].
```

## 1.4. Actividad 4: Cohesión y Acoplamiento – Análisis Comparativo

Compare los dos diseños siguientes y responda las preguntas de análisis:

```
# DISEÑO A — todo en una clase class SistemaAcademico: def
registrar_estudiante(self, nombre, codigo): ... def agregar_nota(self, codigo_est,
curso, nota): ... def calcular_ppa(self, codigo_est): ... def exportar_csv(self,
filename): ... def enviar_reporte(self, email): ... def conectar_bd(self, host,
user, pw): ... # DISEÑO B — responsabilidades separadas class Estudiante: def
agregar_nota(self, nota: Nota): ... def calcular_ppa(self) -> float: ... class
RepositorioEstudiante: def guardar(self, e: Estudiante): ... def buscar(self,
codigo: int) -> Estudiante: ... class ExportadorCSV: def exportar(self,
estudiantes: list[Estudiante], archivo: str): ...
```

| Pregunta de análisis | Su respuesta |
|---|---|
| ¿Cuál diseño tiene mayor cohesión? ¿Por qué? | El diseño B, porque aplica el Principio de Responsabilidad Única. Cada clase tiene un único propósito: **Estudiante** maneja los datos y el PPA. |

| | |
|---|---|
| | **RepositorioEstudiante** se encarga de la persistencia y **ExportadorCSV,** genera los reportes. El Diseño A tiene baja cohesión porque es una clase "Dios" que mezcla lógica de negocio, base de datos, archivos y envío de correos. |
| ¿Qué ocurre con el diseño A si cambia el formato de exportación? | Se obliga a modificar directamente la clase **SistemaAcademico,** lo que pone en riesgo el código existente. Además, esto puede romper de forma no intencionada funciones que no tienen relación con archivos y exige volver a probar y desplegar todo el sistema. |
| ¿Qué principio SOLID viola el Diseño A? | Viola el Principio de Responsabilidad Única, porque la clase tiene múltiples razones para cambiar (lógica de negocio, persistencia, generación de reportes y red). También viola el Principio de Abierto/Cerrado (OCP), ya que para agregar una nueva función hay que modificar la clase en lugar de extenderla. |
| ¿El Diseño B es más fácil de probar unitariamente? ¿Por qué? | Si, porque sus clases están desacopladas. Se pueden probar los métodos de **Estudiante** en memoria de forma instantánea sin necesidad de conectar una base de datos real, enviar correos ni crear archivos en el disco duro. |

## 1.5. Actividad 5: Inicio del Proyecto Integrador – Propuestas y primeras clases

El Proyecto Integrador evolucionará semana a semana hasta la semana 17. Hoy inicia con la propuesta del sistema candidato y el diseño de las primeras clases.

- Elija un sistema candidato del entorno regional o universitario: Ejemplos:
- Sistema de Registro Académico UNA-PUNO | Monitoreo de Sensores loT Lago Titicaca | Gestión de Turismo del Altiplano | Inventario de Postas de Salud Rurales | Biblioteca Digital Regional
- Complete el siguiente formulario de propuesta:

| | |
|---|---|
| **Nombre del Sistema:** | Sistema de Registro Académico UNA-PUNO |
| **Problema que resuelve:** | Ineficiencia en la gestión manual de notas, falta de validación en los límites de calificación y riesgo de inconsistencia de datos por el uso de archivos de hojas de cálculo aislados |
| **Usuarios Principales:** | Estudiantes, Docentes y Administradores de Escuela Profesional |
| **3 clases candidatas iniciales:** | 1. Estudiante<br>2. Curso<br>3. Matrícula |
| URL del repositorio Github: | https://github.com/Dr00pyx/POO2-2026II-Choque-Chaina.git |

# 2. Preguntas de Reflexión

- De todos los problemas identificados en la Actividad 1, ¿Cuál considera el más perjudicial para el mantenimiento del código a largo plazo? Justifique
  El uso de estructuras de datos frágiles y acceso por índices, porque representar los cursos y notas como listas anidadas e itinerar sobre ellas accediendo a posiciones fijas es el fallo más crítico. Si en el futuro el modelo cambia y se decide agregar un dato adicional (como el código del curso o los créditos) en la lista, todo el código existente que depende de esos índices numéricos se romperá, haciendo que el mantenimiento sea altamente propenso a errores.
- Explique con sus propias palabras la diferencia entre un atributo privado (__nombre en Python, nombre_ con acceso private en C++) y un atributo

simplemente con nombre convencional (_nombre). ¿Por qué Python “confía ” en el desarrollo mientras C++ “impone” la restricción?

- En C++ (**private**): El compilador impone un bloqueo estricto. Intentar acceder a un miembro privado desde afuera de la clase genera un error de compilación.
- En Python con **__nombre** (privado por mangling): Python no bloquea el acceso; simplemente transforma internamente el nombre a - **Clase_nombre** para evitar colisiones entre clases, pero sigue siendo accesible.
- En Python con **_nombre** (convención): Es una señal pública de respeto. Indica al programador “este atributo es de uso externo”, pero la síntaxis del lenguaje no aplica ningún tipo de bloqueo ni modificación al nombre.

Python confía porque apuesta por la flexibilidad y la simplicidad. Confía en que los desarrolladores seguirán las convenciones sin necesidad de guardias sintácticos que restrinjan el dinamismo del lenguaje mientras que C++ impone porque prioriza el control estricto de memoria y la robustez en proyectos masivos o de bajo nivel. El bloqueo de compilación evita modificaciones occidentales de la pila o el heap y permite al compilador aplicar optimizadores agresivas.

- En la Actividad 4, el Diseño B tiene más archivos y clases. ¿Por qué, a pesar de eso, es preferible en un proyecto real que crece en el tiempo? ¿En qué contexto podría ser el Diseño A aceptable?

Es preferible porque garantiza la escalabilidad y la mantenibilidad. Al aislar cada responsabilidad en su clase (Estudiante, Repositorio, Exportador):

- Facilita el trabajo en Equipo: Varios desarrolladores pueden trabajar simultáneamente en distintos módulos sin generar conflictos de código.
- Permite la extensión sin riesgo: Es posible agregar nuevas funcionalidades (como exportar a PDF o cambiar de base de datos) modificando únicamente el modulo correspondiente, sin tocar la lógica central del negocio.

- Simplifica las pruebas: Permite realizar pruebas unitarias en cada componente por separado sin necesidad de levantar servicios externos.

El Diseño A es aceptable únicamente en proyecto pequeños y de corta vida útil:

- Script de un solo uso: Automatizaciones o tareas rápidas de limpieza de datos.
- Prototipos o Pruebas de Conceptos (POC): Cuando se requiere validar una idea en tiempo récord para un cliente antes de decidir si se construirá la arquitectura definitiva.
- Entorno académicos iniciales: Para enseñar conceptos básicos de sintaxis sin introducir aún la complejidad del diseño modular.

- ¿Por qué es importante crear el repositorio Git desde la semana 1, aunque el código sea mínimo, en lugar de esperar a tener "algo que mostrar"?
  Es importante crear el repositorio Git desde la Semana 1 por tres razones fundamentales:
  - Trazabilidad del progreso: Registra la evolución del proyecto desde sus cimientos. Permite entender el porqué de cada decisión de diseño a medida que el código crece, facilitando volver a versiones anteriores si un cambio rompe el sistema.
  - Respaldo y prevención de pérdidas: Protege el avance desde el primer día frente a fallos del sistema local, borrados accidentales o problemas en el equipo de desarrollo.
  - Construcción de hábitos y flujo de trabajo: Establece la disciplina de realizar *commits* atómicos, frecuentes y bien documentados desde el inicio, evitando el mal habito de subir un único *commit* masivo al final del proyecto.